

UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

LICENCIATURA EN DESARROLLO Y GESTIÓN DE SOFTWARE



SISTEMAS DE INFORMACIÓN GERENCIAL

LABORATORIO #2

"CHECKSUM o SUMA DE VERIFICACION"

Prof. Ivan Clarence

Integrante:

Gerald Ríos, 8-987-741

2026.10.27

INDICE

CONTENIDO 3
Paso 1: Crear el archivo plano 3
Paso 2: Escribir el nombre y apellido y guardar 3
Paso 3: Calcular el checksum en Defuse Security 4
Paso 4: Guardar el valor SHA-256 5
Paso 5: Modificar el texto y guardar 5
Paso 6: Recalcular el SHA-256 y comparar con el original 6
Paso 7: Devolver el texto a su estado original 8
Paso 8: Recalcular el SHA-256 y comparar con el original 8
Paso 9: Verificar los hashes con Get-FileHash en PowerShell 9
Control: efecto del salto de línea y del BOM 10
CONCLUSION 12
REFERENCIA 13

CONTENIDO

Objetivo: crear un archivo plano con el nombre y apellido (Gerald Rios), obtener su suma de verificación SHA-256 en la página de Defuse Security y comprobar qué pasa con ese valor cuando el texto se modifica y cuando se devuelve a su estado original.

Paso 1: Crear el archivo plano

En la carpeta de evidencias del laboratorio se creó un archivo de texto vacío llamado «GeraldRios.txt» y se abrió con el Bloc de notas de Windows 11. La barra de estado muestra «0 caracteres» y la codificación UTF-8, que es la que el Bloc de notas usa por defecto al guardar.

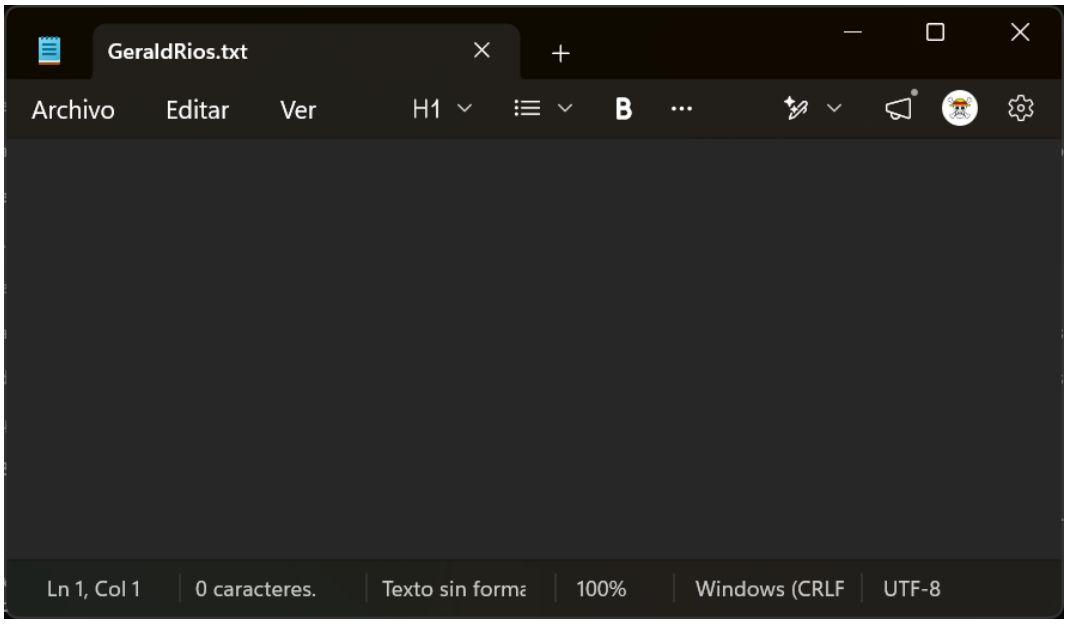


Figura 1. Archivo GeraldRios.txt recién creado, abierto en el Bloc de notas.

Paso 2: Escribir el nombre y apellido y guardar

Se escribió «Gerald Rios» y se guardó con Ctrl + S. El apellido va sin tilde para que cada letra ocupe un solo byte. No se presionó Enter al final, así que el archivo no lleva salto de línea: la barra de estado marca 11 caracteres (6 de «Gerald», el espacio y 4 de «Rios») y el archivo pesa 11 bytes.

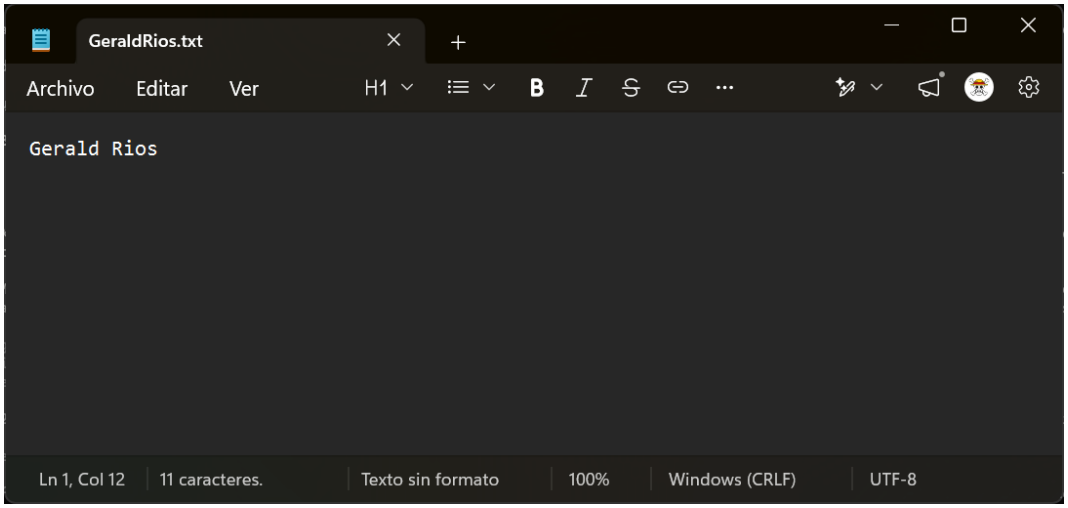


Figura 2. Texto original guardado: 11 caracteres, UTF-8.

Paso 3: Calcular el checksum en Defuse Security

Se entró a https://defuse.ca/checksums.htm#checksums. La página tiene un cuadro para pegar texto («Enter some ASCII or UNICODE text...») y otro para subir un archivo. En el cuadro de texto se escribió el mismo contenido del archivo, «Gerald Rios», sin espacios ni saltos de línea adicionales, y se presionó **Calculate checksums...**. Debajo aparece la sección «Checksums» con el resultado de 61 algoritmos distintos (md5, sha1, sha256, sha512, crc32, whirlpool, entre otros). En la figura se resalta la fila sha256, que es la que pide el laboratorio.

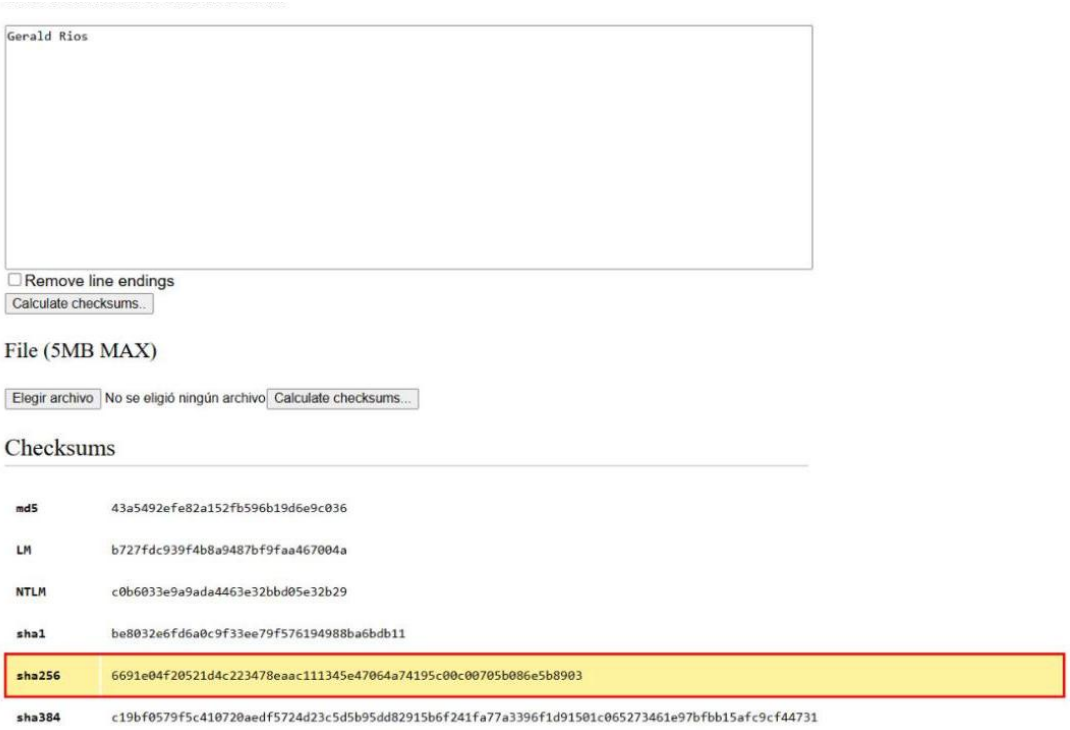


Figura 3. Defuse Security con el texto «Gerald Rios» y su SHA-256 resaltado.

Paso 4: Guardar el valor SHA-256

El valor se copió de la página y se guardó en el archivo «hashes_sha256.txt», junto a las evidencias, para compararlo en los pasos siguientes:

```
Estado 1 (original)  "Gerald Rios"
SHA-256: 6691e04f20521d4c223478eaac111345e47064a74195c00c00705b086e5b8903
```

SHA-256 siempre devuelve 256 bits, que en hexadecimal son 64 caracteres, sin importar si la entrada tiene 11 bytes o varios megabytes.

Paso 5: Modificar el texto y guardar

Se abrió de nuevo GeraldRios.txt, se seleccionó todo el contenido y se reemplazó. El texto quedó «Gerald rios»: la única diferencia con el original es la R del apellido, que ahora está en minúscula. El archivo sigue teniendo 11 caracteres. En la Figura 4 el punto en la pestaña indica que hay cambios sin guardar; en la Figura 5 ya se guardó.

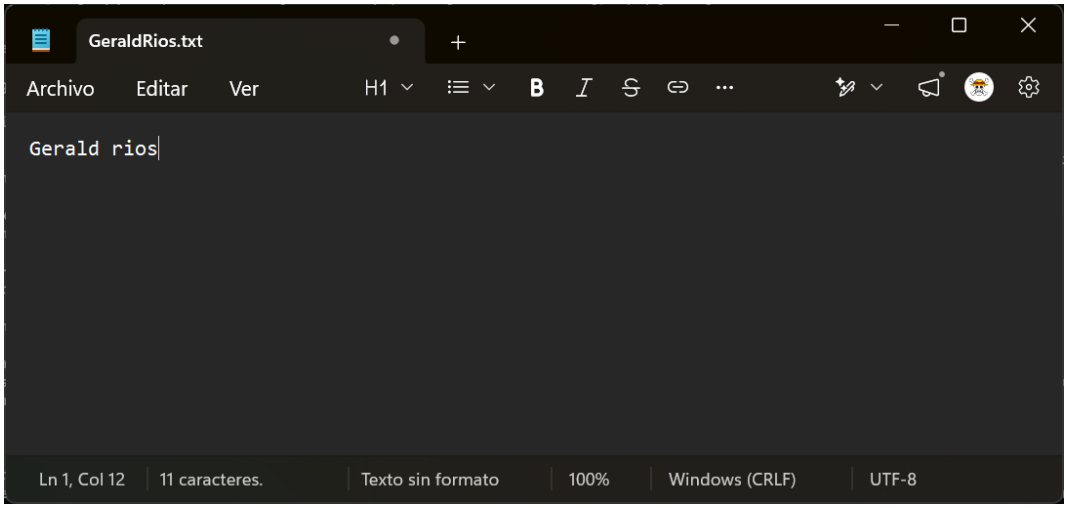


Figura 4. Texto modificado antes de guardar (punto en la pestaña).

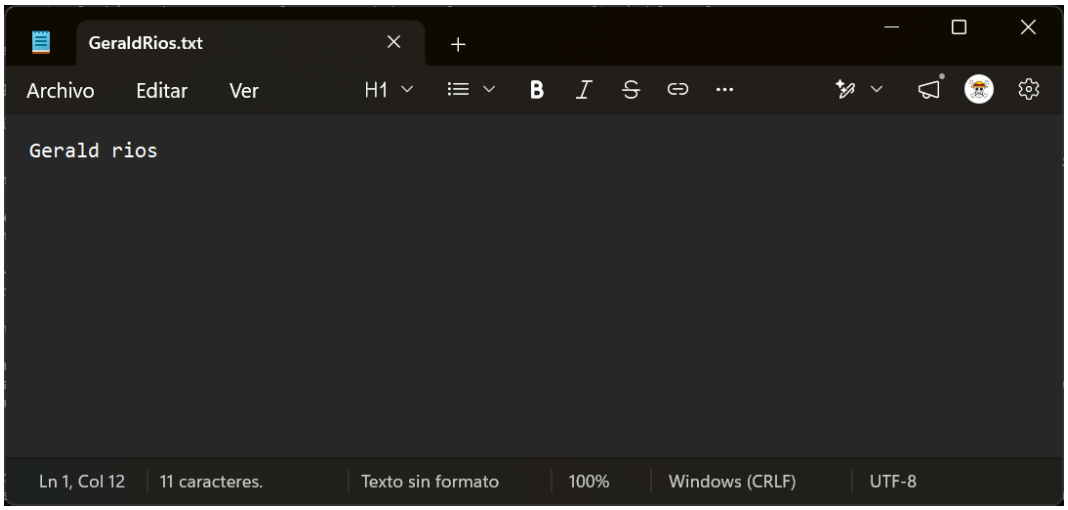


Figura 5. Texto modificado «Gerald rios» ya guardado.

Paso 6: Recalcular el SHA-256 y comparar con el original

En Defuse Security se borró el contenido del cuadro, se escribió «Gerald rios» y se volvió a presionar Calculate checksums..

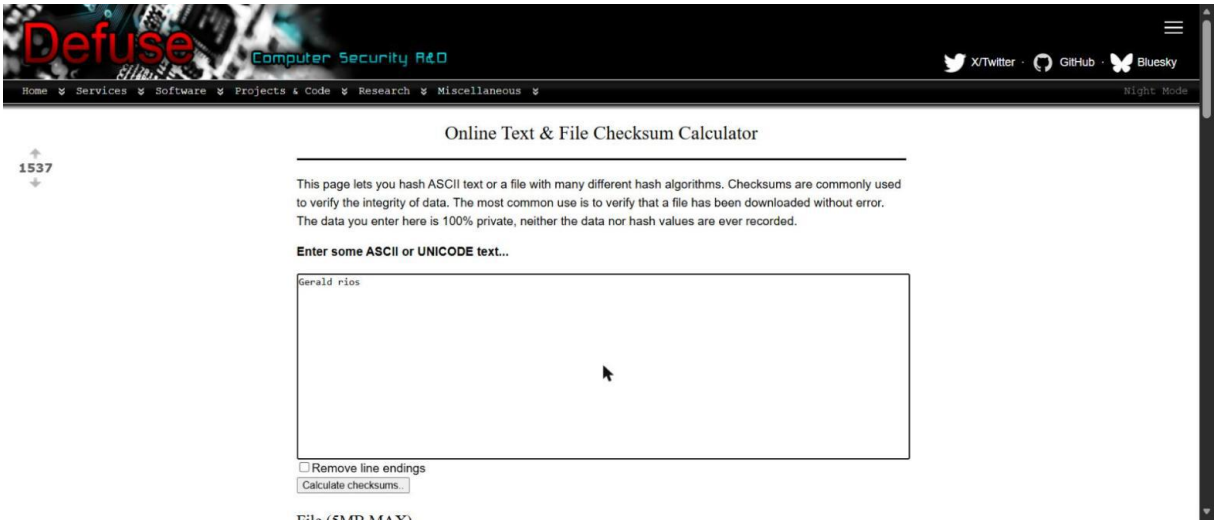


Figura 6. Texto modificado escrito en Defuse Security, antes de calcular.

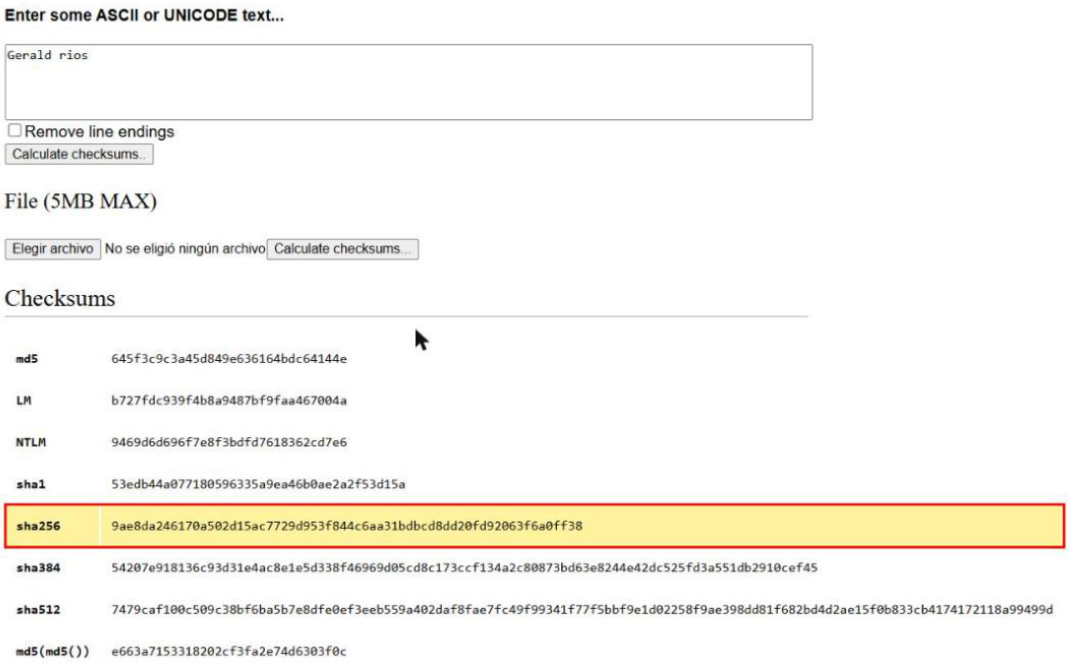


Figura 7. SHA-256 del texto modificado.

```
Original    6691e04f20521d4c223478eaac111345e47064a74195c00c00705b086e5b8903
Modificado  9ae8da246170a502d15ac7729d953f844c6aa31bdbcd8dd20fd92063f6a0ff38
```

Los dos valores no se parecen en nada. Entre «R» (ASCII 82, 01010010) y «r» (ASCII 114, 01110010) solo cambia un bit, y aun así de los 64 caracteres hexadecimales del hash solo uno coincide en la misma posición. Al comparar los dos resultados bit a bit, 136 de los 256 bits son distintos, más o menos la mitad. Esto es lo que se conoce como efecto avalancha: un cambio mínimo en la entrada cambia el hash por completo, y por eso sirve para detectar cualquier alteración.

Paso 7: Devolver el texto a su estado original

Se abrió otra vez el archivo, se reemplazó «Gerald rios» por «Gerald Rios» y se guardó. Quedaron de nuevo 11 caracteres, sin salto de línea al final.

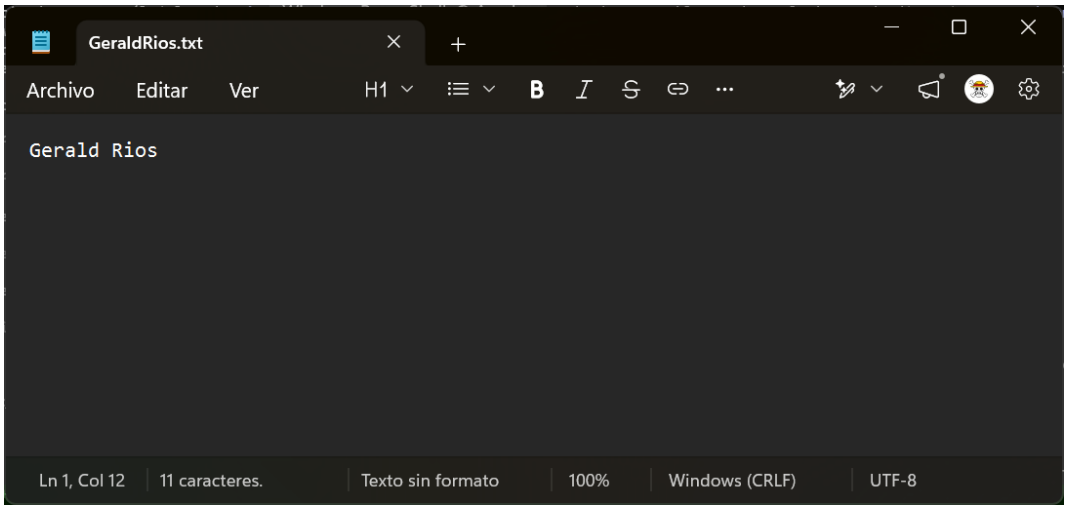


Figura 8. Archivo restaurado a «Gerald Rios» y guardado.

Paso 8: Recalcular el SHA-256 y comparar con el original

En Defuse Security se reemplazó el texto del cuadro por «Gerald Rios» y se calculó otra vez.

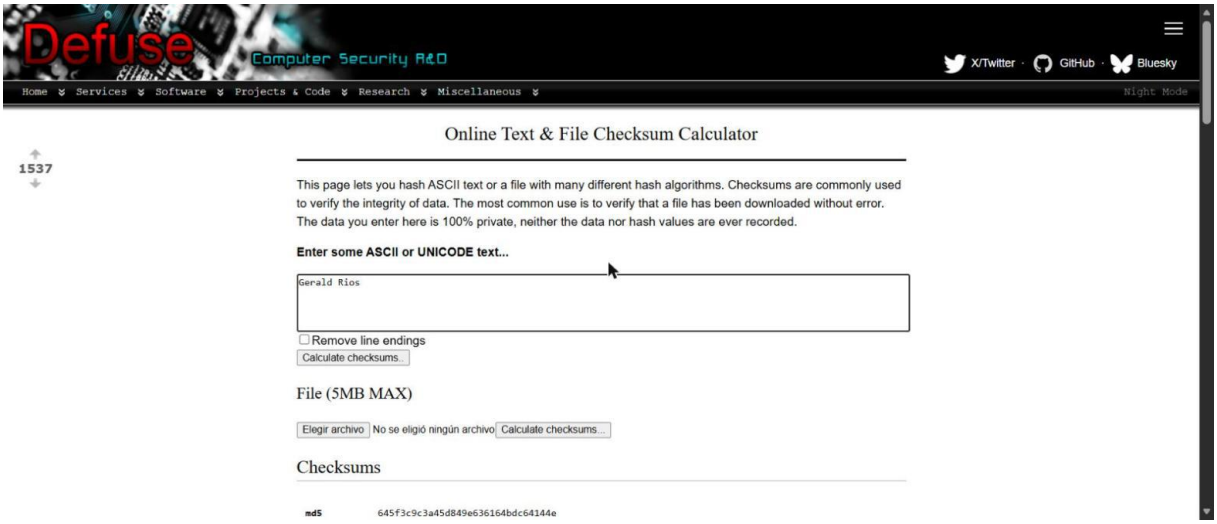


Figura 9. Texto restaurado escrito en Defuse Security, antes de calcular.

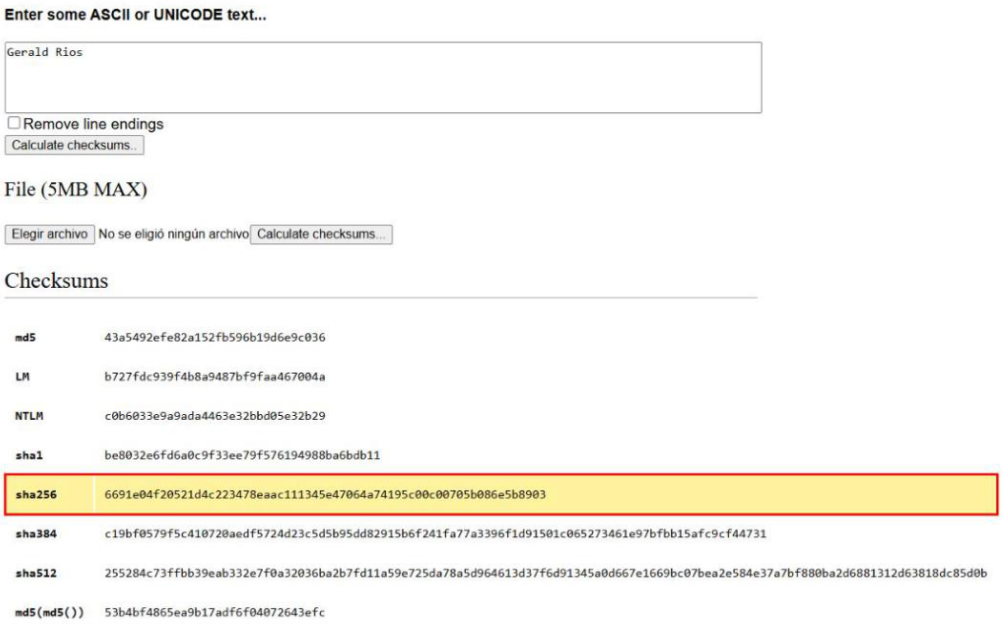


Figura 10. SHA-256 del texto restaurado, igual al del Paso 3.

```
Original     6691e04f20521d4c223478eaac111345e47064a74195c00c00705b086e5b8903
Restaurado   6691e04f20521d4c223478eaac111345e47064a74195c00c00705b086e5b8903
```

El valor coincide carácter por carácter con el del Paso 4. SHA-256 es determinista: el mismo contenido produce siempre el mismo hash, sin importar cuándo se calcule ni cuántas veces se haya editado el archivo antes.

Paso 9: Verificar los hashes con Get-FileHash en PowerShell

Defuse Security calcula el hash del texto que se escribe en la página, no del archivo guardado en disco. Para confirmar que los dos coinciden, después de cada guardado se hizo una copia del archivo (estado1_original.txt, estado2_modificado.txt y estado3_restaurado.txt) y se calculó su SHA-256 en PowerShell con Get-FileHash. También se usó Format-Hex para ver los bytes exactos del archivo original.

```
Get-FileHash .\estado1_original.txt, .\estado2_modificado.txt,
.\estado3_restaurado.txt -Algorithm SHA256
Format-Hex .\estado1_original.txt
```

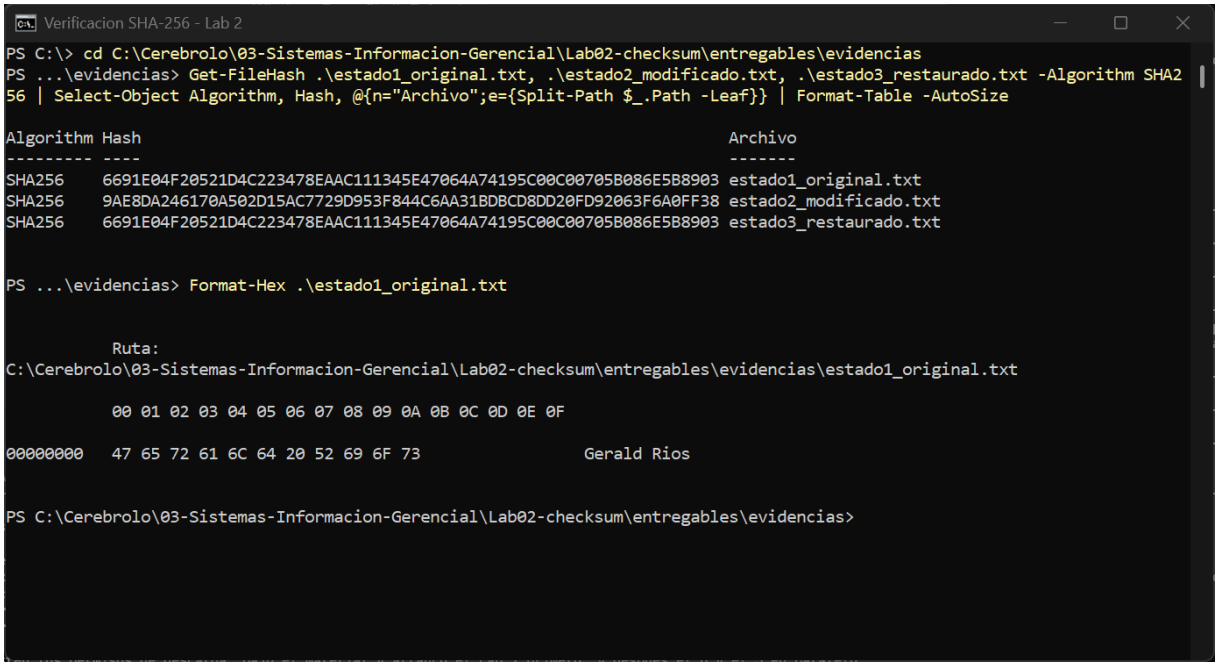


Figura 11. Get-FileHash de los tres estados y bytes del archivo original con Format-Hex.

Los tres valores son los mismos que dio Defuse Security. PowerShell los muestra en mayúsculas y la página en minúsculas, pero en hexadecimal «E» y «e» valen lo mismo (14), así que el número es idéntico. Format-Hex confirma que el archivo tiene 11 bytes justos, 47 65 72 61 6C 64 20 52 69 6F 73, sin la marca BOM de UTF-8 (EF BB BF) al inicio y sin salto de línea (0D 0A) al final.

Control: efecto del salto de línea y del BOM

El hash del archivo coincide con el del texto pegado solo porque el archivo no tiene bytes extra. Para dejarlo comprobado se tomaron los 11 bytes del archivo original y se calculó el SHA-256 de tres variantes en memoria: tal cual, con un salto de línea CRLF al final y con el BOM de UTF-8 al inicio.

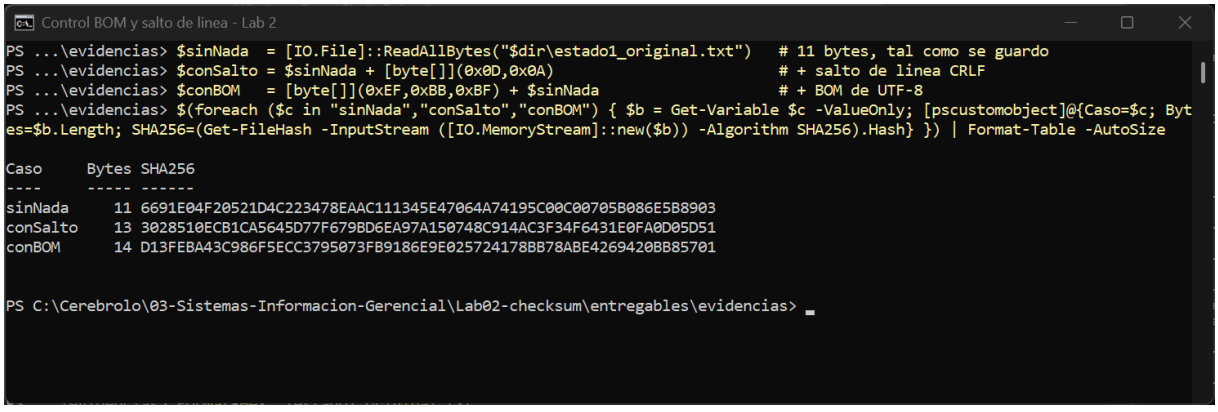


Figura 12. El mismo texto con salto de línea o con BOM produce otro hash.

Con el salto de línea el archivo pasa a 13 bytes y el hash cambia a 3028510e…; con el BOM pasa a 14 bytes y cambia a d13feba4…. Si el archivo se hubiera guardado con Enter al final o con «UTF-8 con BOM», Get-FileHash no habría coincidido con Defuse Security aunque en pantalla se viera el mismo «Gerald Rios». Por eso en el Paso 2 se guardó sin salto de línea y se verificó la codificación.

Tabla resumen

Estado	Texto	SHA-256	¿Igual al original?
1. Original	Gerald Rios	6691e04f20521d4c223478eaac111345 e47064a74195c00c00705b086e5b8903	—
2. Modificado	Gerald rios	9ae8da246170a502d15ac7729d953f84 4c6aa31bdbcd8dd20fd92063f6a0ff38	No
3. Restaurado	Gerald Rios	6691e04f20521d4c223478eaac111345 e47064a74195c00c00705b086e5b8903	Sí

CONCLUSION

La suma de verificación SHA-256 funciona como una huella del contenido. Al cambiar una sola letra del archivo, de «Rios» a «rios», el hash cambió casi por completo: 136 de sus 256 bits fueron distintos. Al devolver el texto a «Gerald Rios», el valor volvió a ser idéntico al del principio. Eso significa que el hash depende únicamente del contenido y no del nombre del archivo, de la hora en que se guardó ni de las ediciones intermedias.

La verificación con Get-FileHash dio los mismos tres valores que Defuse Security, pero dejó claro un detalle práctico: lo que se compara son bytes, no lo que se ve en pantalla. Un salto de línea al final o el BOM de UTF-8 son invisibles en el Bloc de notas y aun así cambian el hash. En un sistema de información esto es lo que permite comprobar que un archivo descargado, un respaldo o un documento enviado no fue alterado: basta con recalcular el hash y compararlo con el publicado.

REFERENCIA

Clarence, I. (2026). Lab 2 CHECKSUM o SUMA DE VERIFICACION [Material del curso]. Universidad Tecnológica de Panamá, Virtual-UTP.

Defuse Security. (s. f.). Online text & file checksum calculator. https://defuse.ca/checksums.htm

Microsoft. (2022). Get-FileHash (Microsoft.PowerShell.Utility). Microsoft Learn. https://learn.microsoft.com/es-es/powershell/module/microsoft.powershell.utility/get-filehash

National Institute of Standards and Technology. (2015). Secure Hash Standard (SHS) (FIPS PUB 180-4). https://doi.org/10.6028/NIST.FIPS.180-4